로봇 알고리즘 프로젝트 1주차 보고서

프로젝트 주제 GPS와 칼만 필터·파티클 필터를 이용한 캠퍼스 위치 추정

팀 번호 20팀

팀원 김정민, 안준성

제출일 2026-11-09

GitHub 저장소

https://github.com/Wenric-min/RobotAlgorithm-GPS-Localization-team20

---

2. 전공 배경지식

2.1 경로 계획에서 샘플링이 필요한 이유

로봇의 경로 계획은 Start state에서 Goal state까지 장애물과 충돌하지 않는 경로를 찾는 문제이다. 이때 로봇의 위치와 자세, 관절각 등으로 구성되는 공간을 Configuration Space라고 한다. Configuration Space는 일반적으로 연속적이며, 로봇의 자유도가 증가하면 공간의 차원도 높아진다. 따라서 공간 전체를 세밀하게 표현하고 가능한 상태와 연결을 모두 조사하는 방식은 많은 계산량과 메모리를 요구한다. 특히 장애물이 복잡한 환경에서는 충돌 없이 이동할 수 있는 Free Configuration Space를 명시적으로 표현하는 것 자체가 어렵다.

샘플링 기반 경로 계획은 이러한 어려움을 줄이기 위해 Configuration Space에서 유한한 수의 상태를 선택하고, 선택한 state와 그 사이의 연결이 충돌 없이 유효한지 검사한다. 이후 유효한 state들을 그래프 또는 트리로 연결하여 Start state에서 Goal state로 이어지는 경로를 탐색한다. 즉, 연속 공간 전체를 직접 다루는 대신 샘플을 통해 공간의 연결 관계를 근사하는 방식이다.

강의자료에서 다룬 샘플링 기반 방법은 다음과 같다.

방법	기본 원리	주요 특징
PRM(Probabilistic Roadmap)	Free C-Space에서 상태를 샘플링하고, 충돌 없이 연결할 수 있는 상태들을 연결하여 로드맵을 만든다.	같은 환경에서 여러 Starting Point·target point에 대한 경로 질의를 처리하는 데 적합하다.
RRT(Rapidly-exploring Random Tree)	샘플을 선택하고 기존 트리에서 가까운 상태를 찾아 샘플 방향으로 트리를 확장한다.	하나의 Start state와 Goal state가 주어진 단일 질의 문제에 활용된다.

방법	기본 원리	주요 특징
EST(Expansive Space Tree)	탐색이 덜 이루어진 영역으로 트리를 확장하도록 샘플링을 수행한다.	Free C-Space의 다양한 영역을 탐색하는 단일 질의 계획 방법이다.

샘플링은 복잡한 공간의 탐색을 계산 가능한 규모로 만드는 데 유용하지만, 유한한 수의 샘플만으로 경로 발견을 항상 보장하지는 않는다. 예를 들어 좁은 통로에 샘플이 충분히 생성되지 않으면 실제로 경로가 존재해도 발견하지 못할 수 있다. 따라서 샘플 수와 샘플링 전략은 경로 발견 가능성과 계산 시간에 영향을 준다.

2.2 칼만 필터(KF)의 개념과 적용 상황

칼만 필터(Kalman Filter)는 noise가 포함된 measurement와 시스템의 동작 모델을 이용하여 시간에 따라 변하는 state를 재귀적으로 추정하는 알고리즘이다. 여기서 상태는 로봇의 위치나 속도처럼 알고자 하는 물리량을 의미한다. 칼만 필터는 상태의 추정값뿐 아니라 추정의 불확실성도 함께 관리하며, 이를 각각 평균 벡터와 공분산 행렬로 표현한다.

기본 칼만 필터는 state 변화와 관측이 선형 모델로 표현되고, 초기 state가 Gaussian distribution을 따르며, Process noise과 Measure nosie가 서로 독립적인 영평균 Gaussian nosie이라는 가정을 사용한다. 강의자료의 상태 모델과 관측 모델은 다음과 같다.

$$x_t = A_t x_{t-1} + B_t u_t + \epsilon_t$$

$$z_t = C_t x_t + \delta_t$$

여기서 x_t는 state, u_t는 control input, z_t는 measurement이다. A_t는 state transition을 나타내는 행렬, B_t는 control input의 영향을 나타내는 행렬, C_t는 state를 measurement로 연결하는 행렬이다. 또한 ϵ_t는 Process noise, δ_t는 Measure nosie이다.

칼만 필터의 동작은 **예측(Prediction)**과 **보정(Correction)**의 반복으로 이루어진다.

예측 단계에서는 이전 시점의 state estimate과 control input을 이용하여 현재 상태를 예측한다. 이때 Process noise을 고려하여 예측의 불확실성도 계산한다. 이후 보정 단계에서는 실제 센서 measurement와 예측 state로부터 예상한 measurement의 차이를 계산한다. 이 차이를 Innovation이라고 하며, 칼만 이득(Kalman Gain)을 곱하여 predicted state를 수정한다.

$$\mu_t = \bar{\mu}_t + K_t(z_t - C_t\bar{\mu}_t)$$

위 식에서 $\bar{\mu}_t$는 예측한 state의 평균, μ_t는 관측을 반영한 state의 평균, K_t는 칼만 이득이다. 칼만 이득은 예측과 관측의 불확실성을 고려하여 새로운 관측을 얼마나 반영할지 결정한다.

칼만 필터는 **센서 측정값에 오차가 있고, 시스템의 state를 시간에 따라 지속적으로 추정해야 할 때** 필요하다. 예를 들어 캠퍼스에서 이동하는 사람의 위치를 추정한다고 하자. GPS 측정값에는 오차가 있고, 이동 모델로 예측한 위치도 실제 이동과 차이가 생길 수 있다. 이러한 상황에서 칼만 필터는 이동 모델의 예측과 새로운 GPS 관측을 결합하여 위치와 속도를 추정하는 데 활용될 수 있다. 이는 적용을 설명하기 위한 예시이며, 실제 프로젝트에서는 상태와 관측 모델을 구체적으로 정의해야 한다.

2.3 파티클 필터(PF)의 개념과 적용 상황

파티클 필터(Particle Filter)는 여러 개의 샘플(Particle)을 이용하여 시스템의 state에 대한 확률분포를 표현하고 추정하는 알고리즘이다. 각각의 파티클은 로봇이 존재할 수 있는 하나의 state를 나타내며, 여러 파티클의 분포를 통해 현재 state에 대한 가능성과 불확실성을 표현할 수 있다.

칼만 필터가 평균과 공분산을 이용하여 Gaussian distribution 형태로 상태를 표현하는 것과 달리, 파티클 필터는 여러 개의 샘플을 이용하므로 서로 다른 위치에 대한 가능성을 동시에 나타낼 수 있다. 예를 들어 로봇이 비슷한 구조의 여러 위치 중 어디에 있는지 확실하지 않은 상황에서는 파티클을 여러 위치에 분포시켜 각 위치의 가능성을 표현할 수 있다.

일반적인 파티클 필터는 로봇의 이동 정보를 이용하여 파티클의 state를 예측하고, 센서 측정값과의 일치 정도에 따라 파티클마다 Weight를 부여한다. 이후 Weight가 높은 파티클을 더 많이 선택하는 재샘플링(Resampling) 과정을 통해 측정값에 부합하는 상태를 중심으로 확률분포를 갱신한다. 이 과정을 반복하면서 로봇의 state를 추정한다.

따라서 파티클 필터는 초기 위치를 알지 못하는 Global Localization이나 기존의 위치 추정이 잘못되어 현재 위치를 다시 찾아야 하는 Recovery 상황에서 필요하다. 하나의 위치만 가정하지 않고 여러 위치에 대한 가능성을 동시에 고려할 수 있기 때문이다.

2.4 중요도 샘플링과 파티클 필터에서의 역할

Important Sample은 Importance Sampling 과정에서 Proposal Distribution으로부터 추출되어 Target Distribution에 대한 Importance Weight가 부여된 샘플을 의미한다. Target Distribution은 추정하려는 목표 확률분포이며, Proposal Distribution은 실제로 샘플을 추출하는 데 사용하는 확률분포이다. Importance Sampling은 목표 확률분포에서 직접 샘플을 추출하기 어려울 때 다른 분포를 이용해 샘플을 생성하고, 그 차이를 Weight로 보정하는 방법이다.

제안 분포에서 샘플을 추출하면 추출된 샘플의 분포가 목표 확률분포와 다를 수 있다. 이를 그대로 사용하면 목표 확률분포를 제대로 나타내지 못하므로 각 샘플에 Importance

Weight를 부여한다. Weight는 해당 샘플에서 Target Distribution과 Proposal Distribution의 확률밀도 비율에 비례한다.

$$w_t^{[i]} \propto \frac{p(x_t^{[i]})}{q(x_t^{[i]})}$$

여기서 $x_t^{[i]}$는 i번째 샘플의 state, $p(x_t)$는 Target Distribution, $q(x_t)$는 Proposal Distribution, $w_t^{[i]}$는 해당 샘플의 Importance Weight이다. Weight가 클수록 그 샘플이 목표 확률분포를 표현할 때 더 큰 영향을 미친다.

파티클 필터에서는 로봇의 이동 모델을 이용하여 파티클을 생성한 뒤 센서 측정값을 바탕으로 각 파티클의 Weight를 계산할 수 있다. 센서 측정값과 잘 부합하는 파티클은 큰 Weight를 받고, 그렇지 않은 파티클은 작은 Weight를 받는다. 이러한 Weight를 상태 추정과 재샘플링에 활용하면 실제 state일 가능성이 높은 영역이 더 많이 반영된다. 따라서 Important Sample은 파티클 필터가 샘플들의 중요도 차이를 고려하여 state의 확률분포를 추정하는 데 필요하다.

2.5 Localization의 목적과 현실 적용 사례

Localization은 로봇이 센서 관측과 이동 정보를 이용하여 지도 또는 기준 좌표계에서 자신의 위치와 자세를 추정하는 문제이다. 강의자료에서는 이를 “세계 안에서 나는 어디에 있는가”라는 질문으로 설명하며, 센서 관측을 환경 모델과 연결하여 로봇의 위치를 계산하는 과정으로 제시한다.

로봇이 목표 지점까지 이동하려면 목표 위치뿐 아니라 현재 자신의 위치도 알아야 한다. 경로가 주어져 있더라도 로봇이 경로의 어느 지점에 있는지 알 수 없다면 올바른 이동 방향과 제어 입력을 결정하기 어렵다. 또한 실제 환경에서는 센서 잡음과 이동 오차 때문에 명령한 이동 결과와 실제 위치가 달라질 수 있다. 따라서 로봇은 이동하면서 새로운 관측을 반영하여 자신의 위치를 계속 추정해야 한다.

Localization 문제는 초기 위치에 대한 지식에 따라 구분할 수 있다. 초기 위치를 알고 그 이후의 위치 변화를 추정하는 경우는 위치 추적(Tracking)이고, 초기 위치를 모르는 상태에서 지도 전체를 대상으로 위치를 찾는 경우는 전역 위치추정(Global Localization)이다. 기존에 알고 있던 위치가 잘못된 경우에는 재위치추정(Re-localization)이 필요하다.

현실의 대응 사례로는 캠퍼스 위치 추정, 실내 배송 로봇, 자율주행 차량 등을 들 수 있다. 캠퍼스 위치 추정에서는 GPS와 이동 정보를 이용하여 사용자가 현재 지도상의 어느 위치에 있는지 추정한다. 실내 배송 로봇은 주변 벽이나 문 등에 대한 센서 관측을 지도와 비교하여 복도에서 자신의 위치를 파악하고, 이를 바탕으로 목적지까지 이동한다. 자율주행 차량에서도 도로 지도와 센서 정보를 연결하여 현재 위치를 추정하는 과정이 주행 판단의 기초가 된다.

한편 Mapping은 주변 환경이 어떻게 생겼는지를 추정하는 문제이며, Localization과 Mapping을 동시에 수행하는 문제를 SLAM(Simultaneous Localization and Mapping)이라고 한다. 따라서 Localization은 자율 이동을 위한 기초 기능이면서, 지도 작성과 결합하여 미지의 환경에서 이동하는 데에도 중요한 역할을 한다.

2.6 Occupancy Grid Map의 개념과 생성 방법

Occupancy Grid Map은 주변 환경을 일정한 크기의 격자(Cell)로 나누고, 각 Cell에 장애물이 존재할 확률을 나타내는 지도이다. 이를 통해 장애물이 존재하는 공간(Occupied), 장애물이 없는 빈 공간(Free), 아직 관측되지 않은 공간(Unknown)을 구분해 표현할 수 있다.

Occupancy Grid Map을 생성하기 위해서는 로봇의 현재 위치와 방향, LiDAR 등의 센서 측정값이 필요하다. LiDAR는 주변 물체까지의 거리와 방향을 측정하므로, 측정 당시 로봇의 위치를 기준으로 센서가 감지한 장애물이 지도상의 어느 Cell에 해당하는지 결정할 수 있다.

먼저 센서 빔이 장애물에 도달하기 전까지 통과한 Cell들은 빈 공간(Free)으로 반영하고, 빔의 끝에서 장애물이 감지된 Cell은 점유 공간(Occupied)으로 반영한다. 반면 센서가 관측하지 못한 영역은 장애물의 유무를 확정할 수 없으므로 Unknown 상태로 남겨 둔다. 이때 단일 측정값에도 오차가 있을 수 있으므로, 각 Cell을 단순히 장애물의 유무로만 표시하지 않고 점유 확률로 나타낸다.

로봇이 이동하면서 새로운 센서 측정값을 얻으면 이전에 계산한 점유 확률을 새로운 관측에 따라 갱신한다. 동일한 공간을 여러 번 관측할수록 해당 Cell이 비어 있는지 또는 장애물이 존재하는지에 대한 정보가 누적된다. 이러한 과정을 반복하여 로봇이 주행하는 환경의 Map을 점차 생성하고 갱신한다.